JAVA PROGRAMMING

Case Study 176 - FOOD DELIVERY BILL SYSTEM (GUI Version)

Name	Mohammed Saad Mulla
Roll No	150096725112
Cohort	Sam Altman
Program	B.Tech CSE
B.Tech Year	2025-2029
Subject	Java Programming
Semester	3
Case Study No	176

1. Introduction

Food delivery apps are used by a lot of people every day. When a customer places an order, the system has to add up the price of every item, apply an offer if there is one, add tax and the delivery charge, and then show one final amount to pay. If this is done by hand, mistakes can happen very easily, so a program is needed for it.

In this project I have made a simple GUI based food delivery billing system in Java using Swing. The customer sees a window with a menu and buttons. Items are added one by one by clicking the Add Item button and entering the item number and quantity in dialog boxes. When the order is finished, the Final Bill button shows a complete bill inside the window.

2. Problem Statement

A food delivery service requires a system to calculate food order totals, delivery charges, taxes, and discounts.

3. Objectives

- Display food items.
- Accept customer orders.
- Calculate food cost.
- Add delivery charges and taxes.
- Apply promotional discounts.

4. Java Concepts Used

The case study clearly says which Java topics have to be used. The table below shows each requirement and where it is used in my program.

Concept	Requirement in case study	Where it is used in the program
Arrays	Store menu items and prices	items[], prices[] and quantities[]
Scanner (input)	Accept customer orders	JOptionPane.showInputDialog() inside the readNumber() method. This is the GUI way of taking input, used in place of Scanner
switch	Food item selection	selectFood() method
Loops	Process multiple food items	for loops in showMenu(), calculateFoodCost(), showOrder(), showBill() and clearOrder()
Operators	Calculate the order total	*, +, -, +=, ++, and comparison operators like >=, == and <=
if-else	Apply promotional discounts	calculateDiscount() and calculateDeliveryCharge()
Methods	Handle bill calculation	showMenu(), selectFood(), addItem(), calculateFoodCost(), calculateDiscount(), calculateDeliveryCharge(), calculateTax(), showOrder(), showBill(), clearOrder(), createButton(), readNumber()

5. Modules of the Project

No	Module	Method used	Work done
1	Menu Display	showMenu()	Shows all food items with their prices in the text area
2	Food Selection	selectFood() and addItem()	Takes the item number and quantity from the customer
3	Order Calculation	calculateFoodCost()	Adds up price x

No	Module	Method used	Work done
			quantity for all items
4	Discount and Tax	calculateDiscount(), calculateTax(), calculateDeliveryCharge()	Finds the discount, 5% tax and delivery charge
5	Final Bill	showBill()	Shows the full bill with the amount to pay

6. Menu and Billing Rules

6.1 Menu

Menu items and prices are stored in two arrays. The item at position 0 in the first array matches the price at position 0 in the second array, and so on.

No	Item	Price (Rs.)
1	Burger	120
2	Pizza	250
3	Pasta	180
4	Sandwich	100
5	French Fries	80
6	Cold Drink	60

6.2 Discount

Food cost	Discount
Rs. 1000 or more	20% of food cost
Rs. 500 to Rs. 999	10% of food cost
Less than Rs. 500	No discount

6.3 Delivery charge and tax

- If the food cost is Rs. 500 or more, delivery is free. Otherwise the delivery charge is Rs. 40.
- Tax is 5% and it is calculated on the amount left after the discount is removed.

6.4 Bill formula

Final Amount = (Food Cost - Discount) + Tax + Delivery Charge

The case study does not give exact numbers for the offers, tax and delivery charge, so I have chosen these values myself. They can be changed easily inside the methods if needed.

7. Working of the Program

Flow in short:

Start -> Window opens and menu is shown -> Click Add Item -> Enter item number -> switch finds the item -> Enter quantity -> Add to order -> (repeat for more items) -> Click Final Bill -> Food cost -> Discount -> Tax -> Delivery charge -> Bill shown in the window -> Exit

Step by step:

1. The program starts, the window is created and the menu is shown using showMenu().
2. The customer clicks the Add Item button. The menu is shown again and a dialog box asks for the item number.
3. If the customer presses Cancel or types something that is not a number, an error message is shown (or nothing happens on Cancel) and the order is not changed.
4. The number is sent to selectFood(), which uses a switch statement to find the correct item. If the number is wrong, an error dialog is shown.
5. A second dialog box asks for the quantity. If it is zero or negative, an error dialog is shown.
6. The quantity is added to the quantities[] array for that item and the current order is shown. The customer can click Add Item again as many times as needed, so many items can be ordered.
7. The Show Order button shows the items ordered so far and the food cost. The Clear Order button sets all quantities back to 0.
8. When the customer clicks Final Bill, the program checks if anything was ordered. If not, it says so.
9. Otherwise showBill() calculates the food cost, discount, tax and delivery charge and shows the final bill in the text area.
10. The Exit button (or the window close button) closes the program.

8. Source Code

```
import java.awt.*;
import javax.swing.*;

public class FoodDeliveryBillSystem {

    // Arrays to store menu items, prices and ordered quantities
    static String[] items = {"Burger", "Pizza", "Pasta", "Sandwich", "French Fries",
"Cold Drink"};
    static int[] prices = {120, 250, 180, 100, 80, 60};
    static int[] quantities = new int[items.length];

    static JFrame frame;
    static JTextArea area;

    public static void main(String[] args) {
```

```
        // GUI Setup using Swing
        frame = new JFrame("Food Delivery Bill System");
        frame.setLayout(null);
        frame.getContentPane().setBackground(Color.WHITE);

        // Title
        JLabel titleLabel = new JLabel("FOOD DELIVERY BILL SYSTEM", JLabel.CENTER);
        titleLabel.setBounds(30, 15, 830, 30);
        titleLabel.setFont(new Font("SansSerif", Font.BOLD, 18));
        frame.add(titleLabel);

        // Buttons (6 buttons x 125 wide + 5 gaps x 16 = 830)
        JButton menuButton = createButton("Show Menu", 30, 55, 125, 35);
        JButton addButton = createButton("Add Item", 171, 55, 125, 35);
        JButton orderButton = createButton("Show Order", 312, 55, 125, 35);
        JButton billButton = createButton("Final Bill", 453, 55, 125, 35);
        JButton clearButton = createButton("Clear Order", 594, 55, 125, 35);
        JButton exitButton = createButton("Exit", 735, 55, 125, 35);

        // Text area to show output
        area = new JTextArea();
        area.setEditable(false);
        area.setBackground(Color.WHITE);
        area.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scroll = new JScrollPane(area);
        scroll.setBounds(30, 105, 830, 400);

        frame.add(menuButton);
        frame.add(addButton);
        frame.add(orderButton);
        frame.add(billButton);
        frame.add(clearButton);
        frame.add(exitButton);
        frame.add(scroll);

        // Button actions
        menuButton.addActionListener(e -> showMenu());
        addButton.addActionListener(e -> addItem());
        orderButton.addActionListener(e -> showOrder());
        billButton.addActionListener(e -> showBill());
        clearButton.addActionListener(e -> clearOrder());
        exitButton.addActionListener(e -> System.exit(0));

        frame.setSize(900, 580);
        frame.setLocationRelativeTo(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
```

```
        showMenu();
    }

    // Helper method to create plain buttons (default look, no colors)
    static JButton createButton(String text, int x, int y, int w, int h) {
        JButton btn = new JButton(text);
        btn.setBounds(x, y, w, h);
        btn.setFocusPainted(false);
        btn.setFont(new Font("SansSerif", Font.PLAIN, 13));
        return btn;
    }

    // Helper method to read a number from the user (returns -1 if cancelled or
invalid)
    static int readNumber(String message) {
        String input = JOptionPane.showInputDialog(frame, message);
        if (input == null) {
            return -1;
        }
        try {
            return Integer.parseInt(input.trim());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(frame, "Please enter a valid number!",
"Error", JOptionPane.ERROR_MESSAGE);
            return -1;
        }
    }

    // Module 1: Menu Display
    static void showMenu() {
        String text = "=========== MENU ===========\n";
        for (int i = 0; i < items.length; i++) {
            text += (i + 1) + ". " + items[i] + " - Rs. " + prices[i] + "\n";
        }
        text += "============================\n";
        area.setText(text);
    }

    // Module 2: Food Selection (switch statement)
    static int selectFood(int choice) {
        switch (choice) {
            case 1:
                return 0;
            case 2:
                return 1;
            case 3:
                return 2;
```

```
            case 4:
                return 3;
            case 5:
                return 4;
            case 6:
                return 5;
            default:
                return -1;
        }
    }

    // Take order from the customer
    static void addItem() {
        showMenu();

        int choice = readNumber("Enter item number (1-6):");
        if (choice == -1) {
            return;
        }

        int index = selectFood(choice);
        if (index == -1) {
            JOptionPane.showMessageDialog(frame, "Invalid choice. Please try again.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        int qty = readNumber("Enter quantity for " + items[index] + ":");
        if (qty == -1) {
            return;
        }
        if (qty <= 0) {
            JOptionPane.showMessageDialog(frame, "Quantity must be at least 1.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        quantities[index] += qty;
        showOrder();
    }

    // Module 3: Order Calculation
    static int calculateFoodCost() {
        int total = 0;
        for (int i = 0; i < items.length; i++) {
            total += prices[i] * quantities[i];
        }
        return total;
```

```
    }

    // Module 4: Discount (if-else)
    static double calculateDiscount(int foodCost) {
        if (foodCost >= 1000) {
            return foodCost * 0.20;
        } else if (foodCost >= 500) {
            return foodCost * 0.10;
        } else {
            return 0;
        }
    }

    // Delivery charge (if-else)
    static int calculateDeliveryCharge(int foodCost) {
        if (foodCost >= 500) {
            return 0;
        } else {
            return 40;
        }
    }

    // Tax is 5%
    static double calculateTax(double amount) {
        return amount * 0.05;
    }

    // Show the current order
    static void showOrder() {
        if (calculateFoodCost() == 0) {
            area.setText("No items ordered yet.");
            return;
        }

        String text = "=========== YOUR ORDER ===========\n";
        for (int i = 0; i < items.length; i++) {
            if (quantities[i] > 0) {
                text += String.format("%-14s x %d = Rs. %d\n", items[i],
quantities[i], prices[i] * quantities[i]);
            }
        }
        text += "---------------------------------\n";
        text += "Food Cost so far: Rs. " + calculateFoodCost() + "\n";
        area.setText(text);
    }

    // Module 5: Final Bill
    static void showBill() {
```

```
        int foodCost = calculateFoodCost();
        if (foodCost == 0) {
            area.setText("No items ordered yet.");
            return;
        }

        double discount = calculateDiscount(foodCost);
        double afterDiscount = foodCost - discount;
        double tax = calculateTax(afterDiscount);
        int delivery = calculateDeliveryCharge(foodCost);
        double finalAmount = afterDiscount + tax + delivery;

        String text = "============== FINAL BILL ==============\n";
        for (int i = 0; i < items.length; i++) {
            if (quantities[i] > 0) {
                text += String.format("%-14s x %d = Rs. %d\n", items[i],
quantities[i], prices[i] * quantities[i]);
            }
        }
        text += "----------------------------------------\n";
        text += String.format("%-18s Rs. %d\n", "Food Cost", foodCost);
        text += String.format("%-18s Rs. %.2f\n", "Discount (-)", discount);
        text += String.format("%-18s Rs. %.2f\n", "Tax 5% (+)", tax);
        text += String.format("%-18s Rs. %d\n", "Delivery (+)", delivery);
        text += "----------------------------------------\n";
        text += String.format("%-18s Rs. %.2f\n", "TOTAL TO PAY", finalAmount);
        text += "========================================\n";
        text += "Thank you for ordering!\n";
        area.setText(text);
    }

    // Clear the order
    static void clearOrder() {
        for (int i = 0; i < quantities.length; i++) {
            quantities[i] = 0;
        }
        area.setText("Order cleared.");
    }
}
```

9. Explanation of the Code

9.1 Imports and class

import java.awt.*; and import javax.swing.*; are written so that we can use the GUI classes. From java.awt the program uses Color and Font. From javax.swing it uses JFrame, JLabel, JButton, JTextArea,

JScrollPane and JOptionPane. The whole program is inside the class FoodDeliveryBillGUI, and the file name has to be the same as this class name.

9.2 Arrays and variables

items is a String array that stores the food names, prices is an int array that stores their prices, and quantities is an int array that stores how many of each item the customer ordered. All three are static so every method can use them directly. Because they are in the same order, items[1] is Pizza and prices[1] is 250. The quantities array is made with the same size as items, so every quantity starts at 0. The variables frame (the window) and area (the text box) are also declared as static so that every method can use them.

9.3 main()

The program starts from here. A JFrame object is created as the window, and its layout is set to null so that every component can be placed with exact positions using setBounds(x, y, width, height). The background is set to white. A JLabel is added as the title. Six buttons are created by calling createButton(), and a JTextArea (made non-editable, with a monospaced font so the columns line up) is placed inside a JScrollPane. All of these are added to the frame using frame.add().

Each button is connected to a method using addActionListener() with a lambda expression. For example, menuButton.addActionListener(e -> showMenu()) means that when the button is clicked, showMenu() runs. The Exit button calls System.exit(0), which closes the program. At the end, the window size is set, the window is centered, setDefaultCloseOperation(EXIT_ON_CLOSE) makes the close button stop the program, setVisible(true) shows the window, and showMenu() displays the menu at the start.

9.4 createButton()

This helper method creates a JButton with the given text, position and size, turns off the focus outline, sets a plain font and returns the button. It is used so the same four lines do not have to be repeated for all six buttons.

9.5 readNumber()

This helper method shows an input dialog box using JOptionPane.showInputDialog() and returns what the customer typed as a number. If the customer presses Cancel, the dialog gives null and the method returns -1. Integer.parseInt() converts the text to an int. If the customer types a letter or anything that is not a number, a NumberFormatException happens, which is caught by try-catch. An error message is shown and -1 is returned, so the program does not crash.

9.6 showMenu()

This method has a for loop that goes from 0 to items.length - 1 and adds every item with its price to a String. (i + 1) is printed because arrays start from 0 but the menu should start from 1. At the end, area.setText(text) shows the menu in the text area.

9.7 selectFood()

It takes the menu number as a parameter and uses a switch statement. Case 1 returns index 0, case 2 returns index 1 and so on. The default case returns -1 which means the customer entered a wrong number.

9.8 addItem()

This method runs when the Add Item button is clicked. It shows the menu, then asks for the item number using readNumber(). If the customer cancelled (-1), it stops using return. Then selectFood() is called, and if it gives -1 an error dialog is shown. Then the quantity is asked, and if it is 0 or less an error dialog is shown. Otherwise the quantity is added to the array with quantities[index] += qty and showOrder() displays the order so far. Since += is used, ordering the same item again adds to the old quantity.

9.9 calculateFoodCost()

A variable total starts from 0. The for loop multiplies each price with its quantity and adds it to total using +=. Items that were not ordered have quantity 0 so they add nothing. The method then returns total.

9.10 calculateDiscount()

This is where if-else is used. If the food cost is 1000 or more, 20% is returned as discount. Else if it is 500 or more, 10% is returned. Otherwise the discount is 0. The return type is double because the discount can have decimals.

9.11 calculateDeliveryCharge()

Returns 0 if the food cost is 500 or more (free delivery), otherwise returns 40.

9.12 calculateTax()

Takes an amount and returns 5% of it, which is amount * 0.05.

9.13 showOrder()

If the food cost is 0, it shows "No items ordered yet." Otherwise a for loop adds only those items whose quantity is more than 0, and at the end the food cost so far is shown. String.format is used to line up the columns, where %-14s means text with width 14 (left aligned) and %d is for whole numbers.

9.14 showBill()

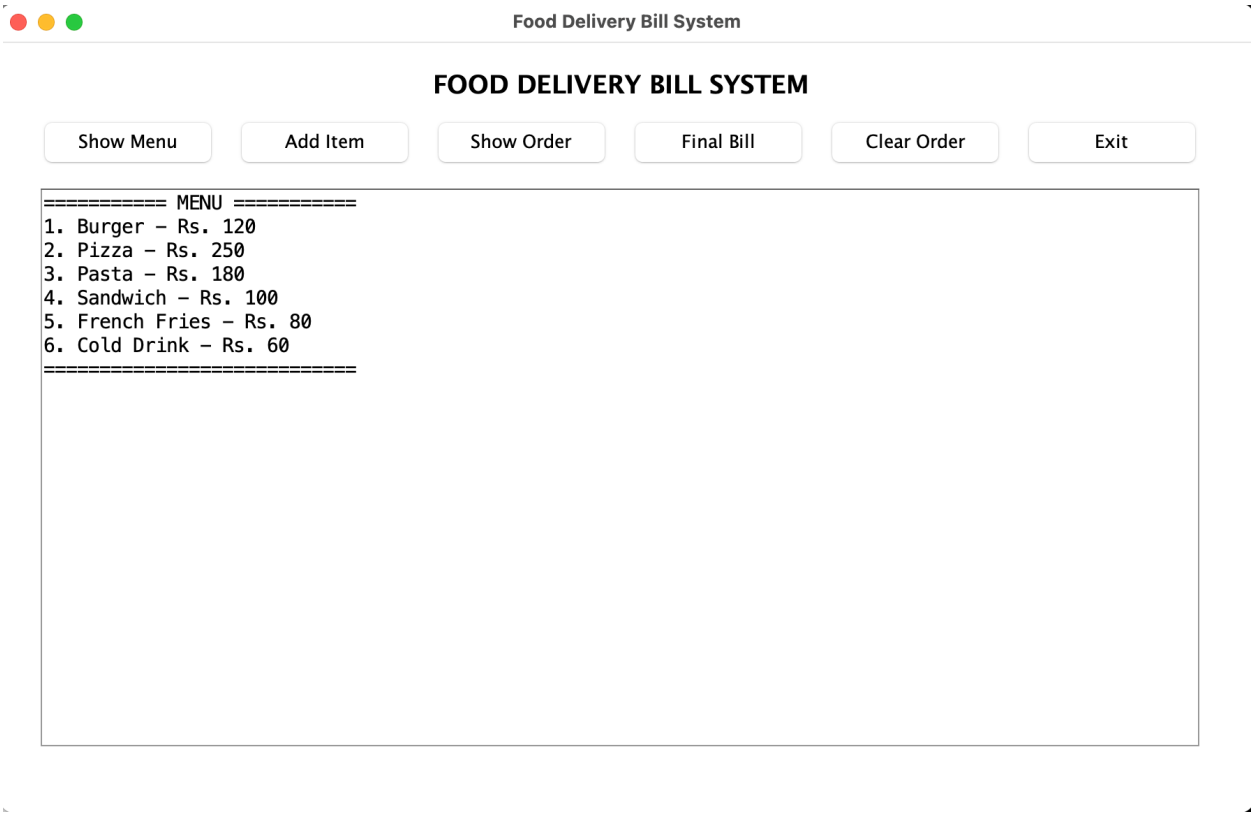
First it checks that something was ordered. Then it calls the other methods one by one to get the food cost, discount, tax and delivery charge. Then it finds the final amount using the bill formula. After that, a for loop adds only those items whose quantity is more than 0, followed by the food cost, discount, tax, delivery and total lines. String.format is used to line up the columns, where %-18s means text with width 18, %d is for whole numbers and %.2f shows a number with 2 decimal places. The full bill text is then shown in the text area with area.setText().

9.15 clearOrder()

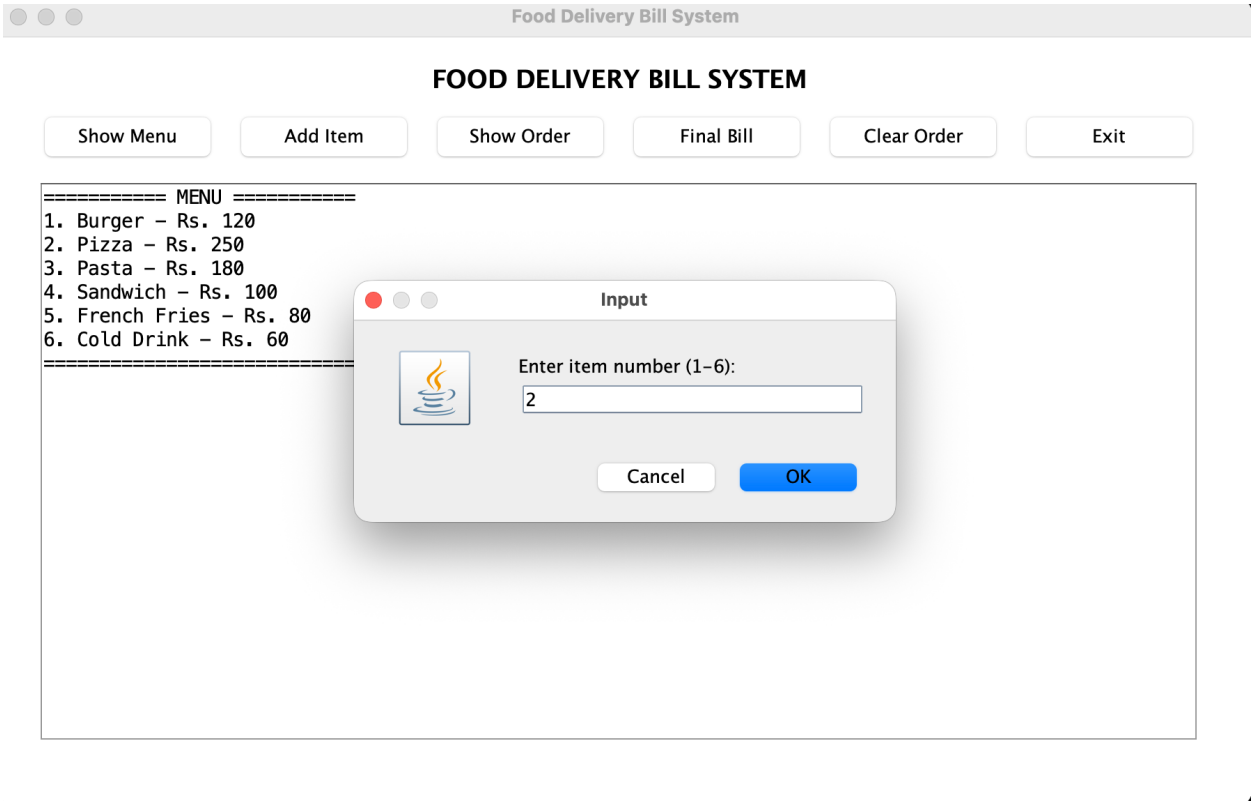
A for loop sets every value of the quantities array back to 0, and the message "Order cleared." is shown. This lets the customer start a new order without restarting the program.

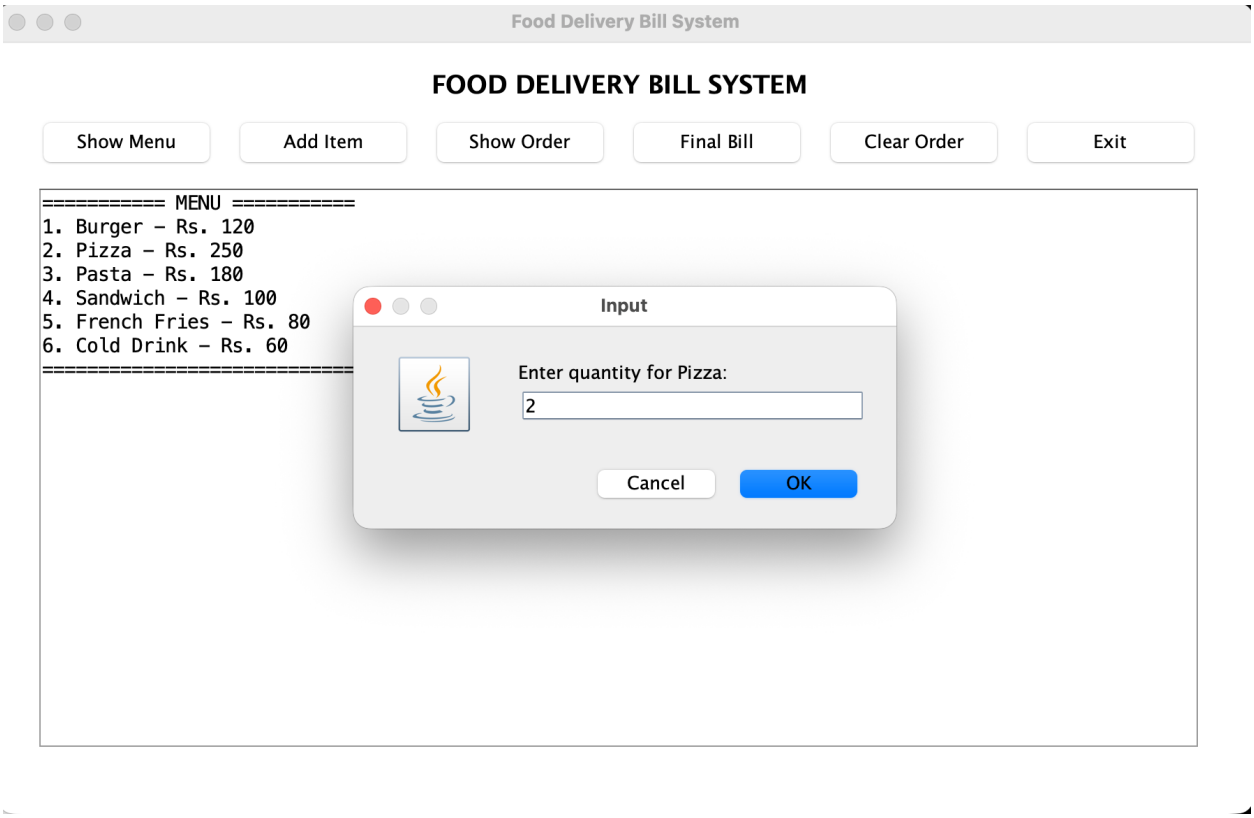
10. Output

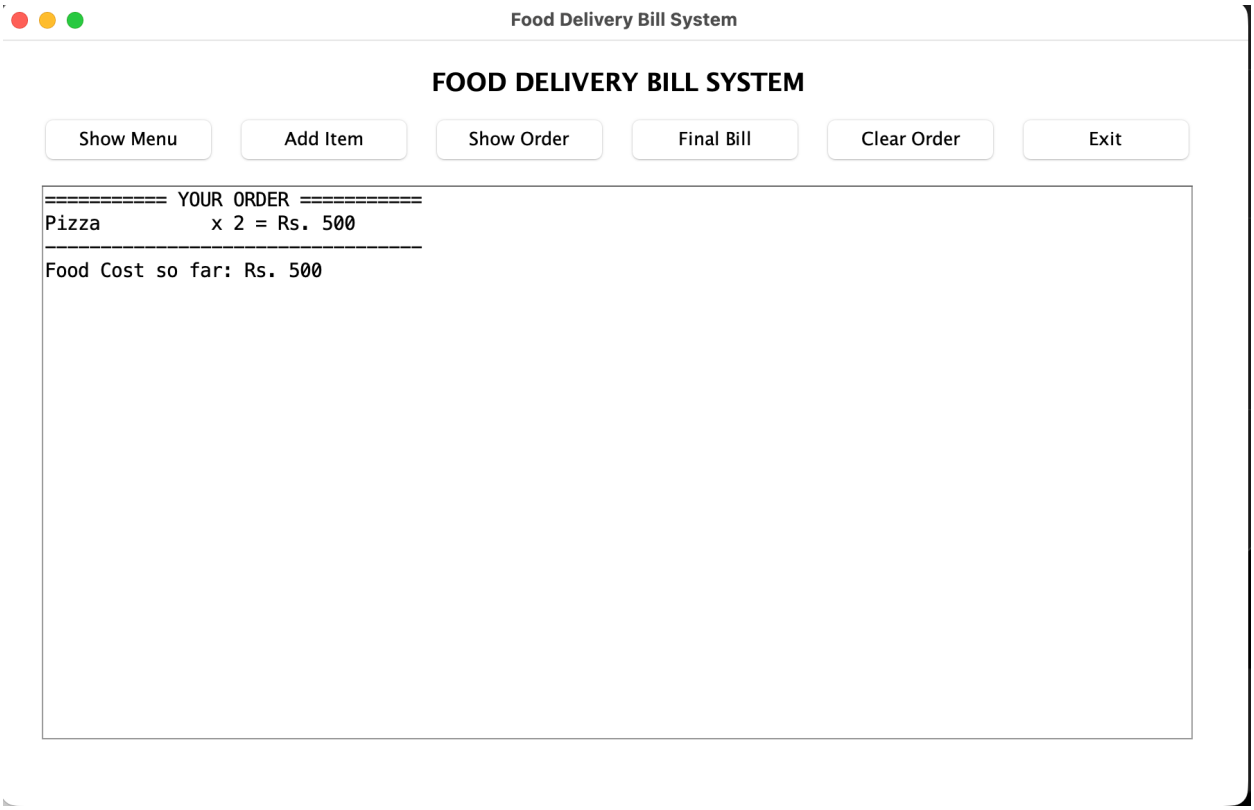
10.1 Menu display



10.2 Adding an item (dialog boxes)

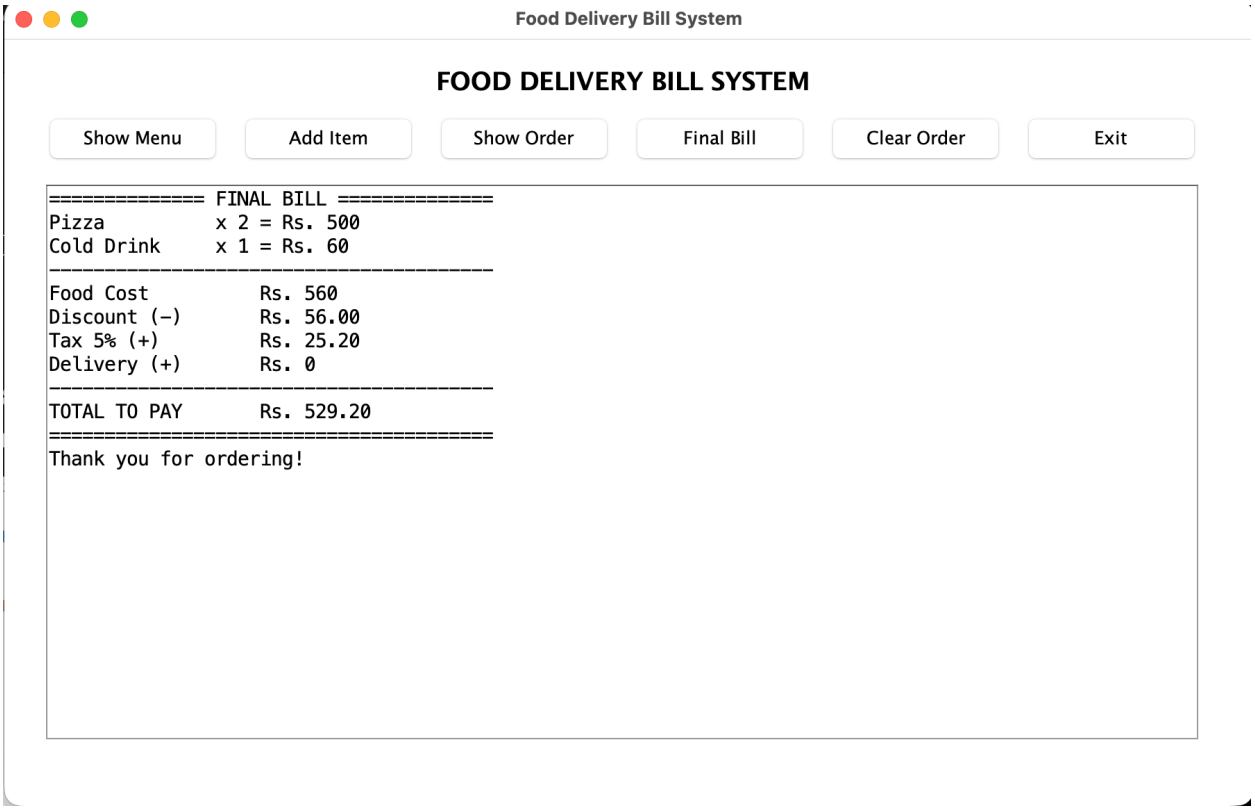



Food Delivery Bill System
FOOD DELIVERY BILL SYSTEM
Show Menu
Add Item
Show Order
Final Bill
Clear Order
Exit
========== MENU ==========
1. Burger – Rs. 120
2. Pizza – Rs. 250
3. Pasta – Rs. 180
4. Sandwich – Rs. 100
5. French Fries – Rs. 80
6. Cold Drink – Rs. 60
==========
Input
Enter quantity for Pizza:
2
Cancel
OK

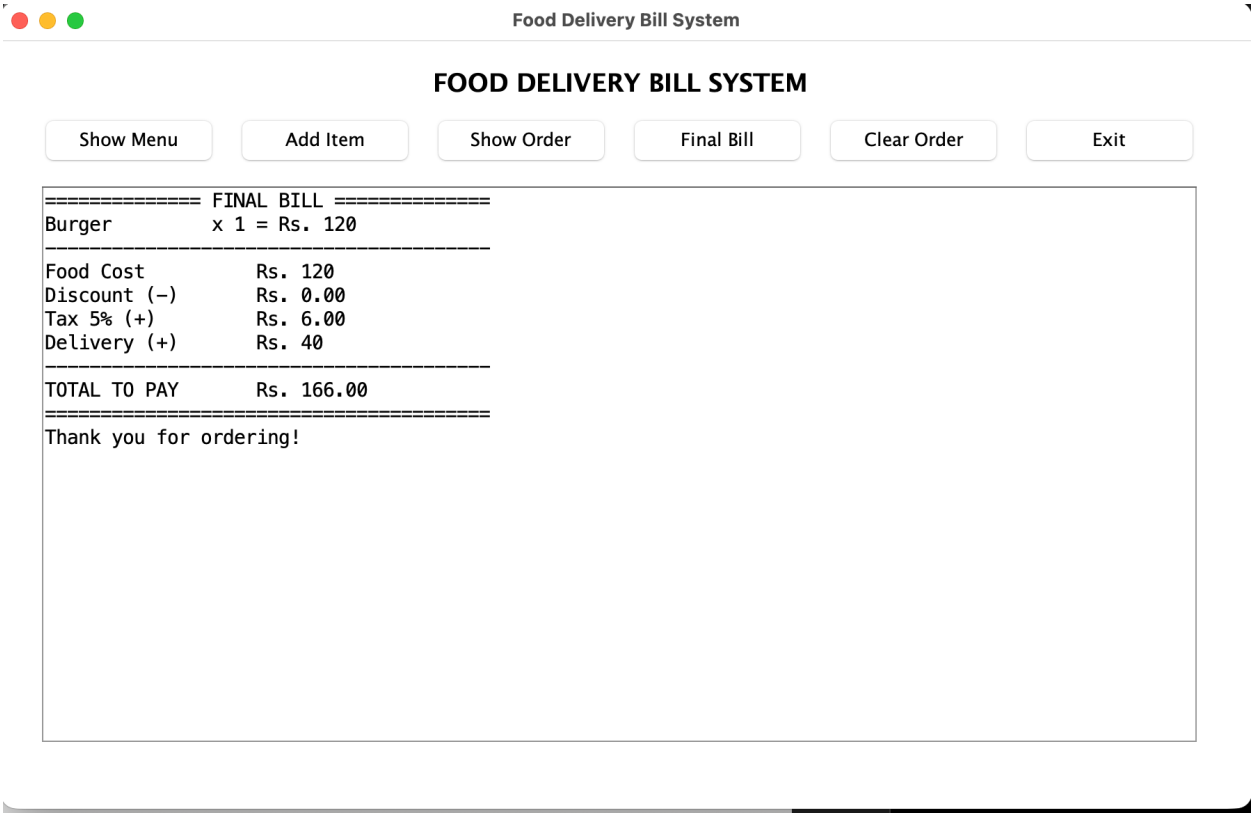

Food Delivery Bill System
FOOD DELIVERY BILL SYSTEM
Show Menu
Add Item
Show Order
Final Bill
Clear Order
Exit
========== YOUR ORDER ==========
Pizza x 2 = Rs. 500

Food Cost so far: Rs. 500

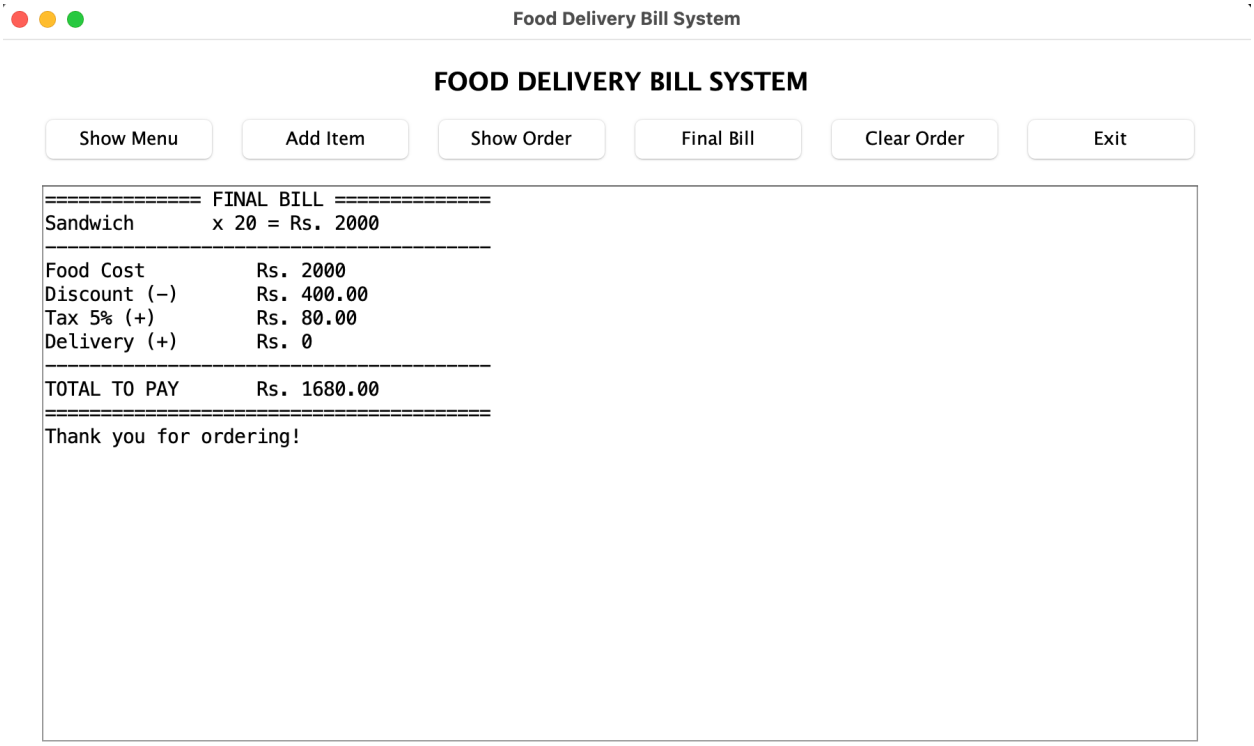
10.3 Test 1: Order with 10% discount and free delivery



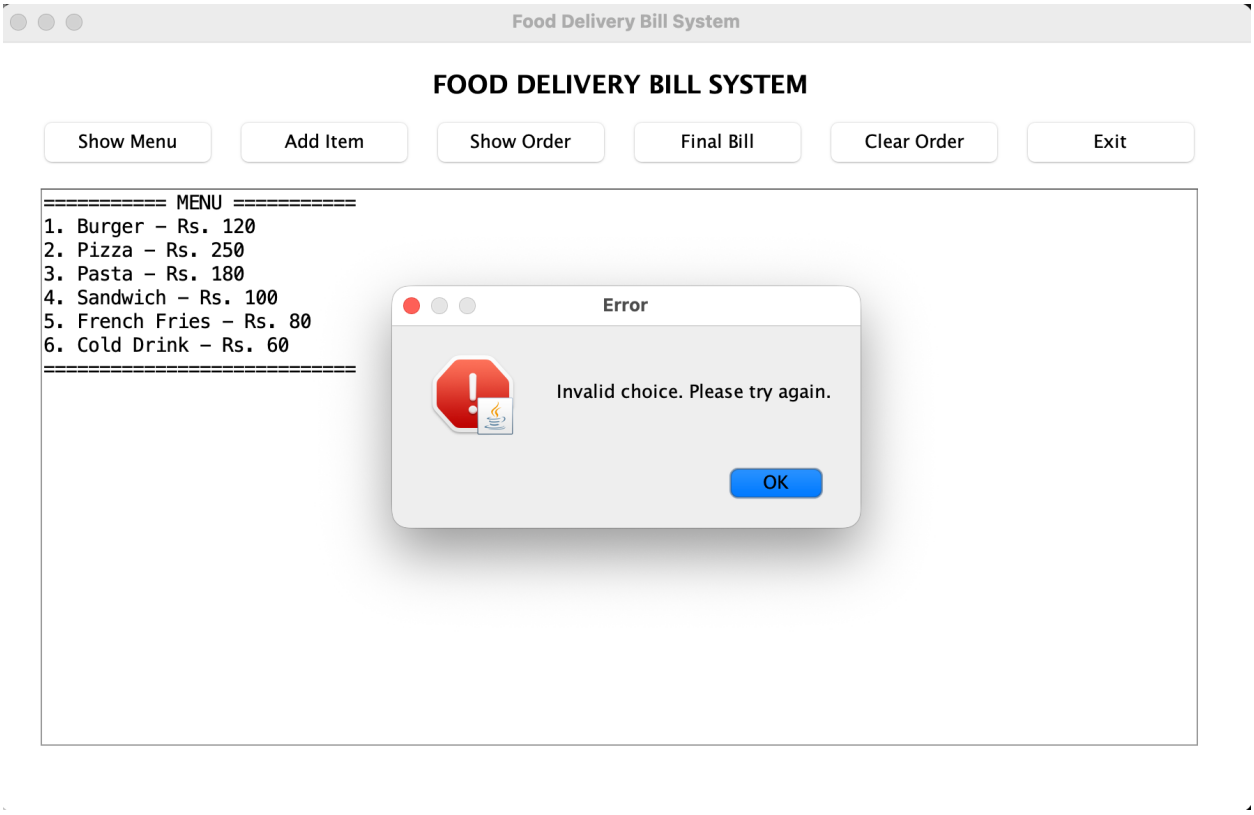
10.4 Test 2: Small order with no discount and delivery charge



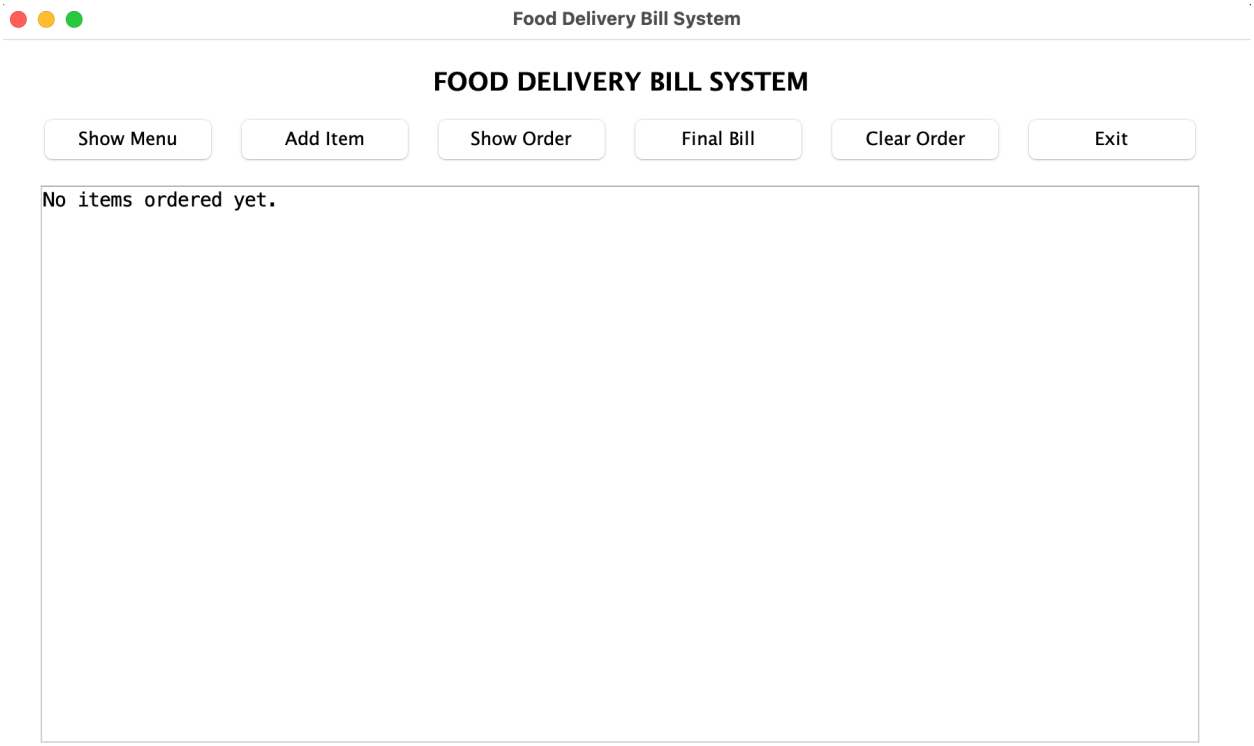
10.5 Test 3: Big order with 20% discount



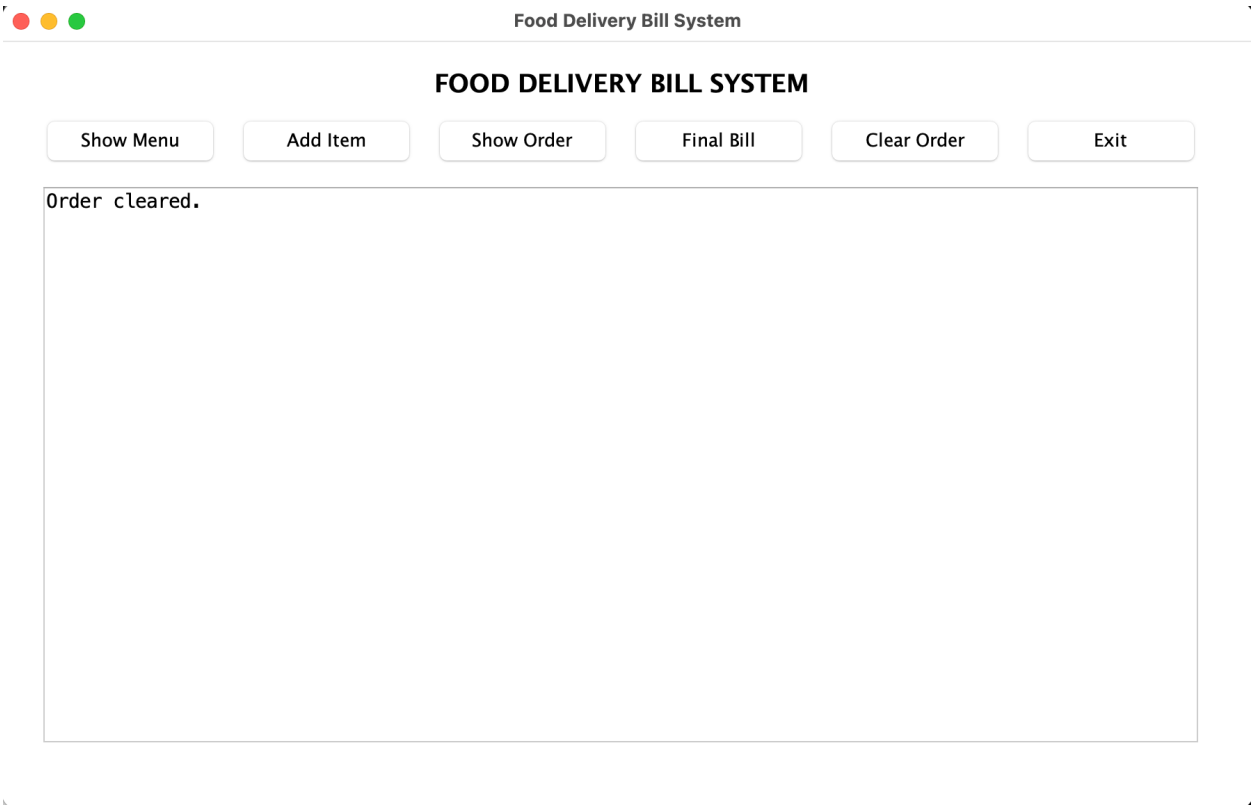
10.6 Test 4: Wrong inputs



10.7 Test 5: No items ordered



10.8 Test 6: Clear Order



11. Conclusion

The Food Delivery Bill System works as the case study asks. It shows the menu, takes orders for many items, finds the food cost, applies a discount, adds tax and delivery charge, and shows a clear final bill in a window. All the required topics, which are arrays, input handling, switch, loops, operators, if-else and methods, are used in a simple way. Splitting the work into methods also made the code easier to write, read and test, and the GUI makes the program easier to use than typing in a console.

12. Limitations and Future Scope

- Input is taken with JOptionPane dialog boxes instead of the Scanner class. A console version using Scanner could also be made.
- Menu items and prices are fixed in the code. In future they could be read from a file or a database.
- A customer name, address and order history can be added.
- The components are placed using fixed pixel positions (null layout), so the window does not resize well. A layout manager like BorderLayout or GridLayout could be used.
- Classes and objects can be used to make a FoodItem class, and the bill could be saved or printed as a file.